SLA-Aware Incident Detection and Alert Routing via MCP in Distributed Systems

Siva Rama Krishna Varma Bayyavarapu
Independent Researcher
Indiana, United States
siva.bayyavarapu@ieee.org

Vijayakumar Venganti
Independent Researcher
San Jose, CA
vijayakumar.venganti@ieee.org

Vishwanath Hiremath
Principal System/Software Engineer
California, United States
hiremath1@ieee.org

***Abstract*—Modern distributed systems generate large volumes of telemetry (metrics, logs, and traces) across interconnected services. In production operations, related failures often surface as multiple fragmented alerts, making it difficult to separate high-impact incidents from background noise. Static threshold-based monitoring can therefore produce excessive false alarms and delay incident triage. We present a Service-Level Agreement (SLA)-aware approach to incident detection and alert routing that integrates three components: (1) an incremental incident-grouping model based on temporal and tag similarity with per-event work $O(K)$; (2) SLA-weighted scoring and capacity-aware routing derived from a utility-based prioritization scheme; and (3) adaptive anomaly detection using cumulative sum (CUSUM) with tunable false-positive control. The design is implemented as a Model Context Protocol (MCP)-based multi-agent pipeline (correlation $\rightarrow$ scoring $\rightarrow$ prioritization $\rightarrow$ routing).**

We evaluate the system primarily on OpenTelemetry-style (OTel-style) microservice telemetry (Track C) over 30 runs with bootstrap 95% confidence intervals and non-parametric statistical tests; we additionally report results on a public key performance indicator (KPI) benchmark (Track A). In Track C, MCP-Agent improves precision over static thresholding (0.171 vs. 0.055) and improves F1 over tag+time grouping (0.282 vs. 0.104), while maintaining a false-positive rate (FPR) below the target budget of 6 alerts/hour. It also improves over a Bayesian changepoint baseline under the same evaluation setting. Scalability experiments (10–200 services) show sub-millisecond p50 latency and throughput above 1800 events/s. The experimental setup is designed to support reproducibility.

***Index Terms*—AIOps, MCP, anomaly detection, CUSUM, event correlation, SLA, alert routing, microservices, observability.**

I. Introduction

Modern distributed infrastructures (microservices, IoT deployments, and multi-cloud systems) require continuous monitoring to detect failures before they affect users. Conventional observability tools [1]–[5] collect metrics and logs but typically rely on predefined rules or static thresholds. Under dynamic workloads, such approaches often produce excessive false alarms and fragmented alerts, increasing operator burden and slowing incident resolution. As a result, production environments increasingly incorporate data-driven Artificial Intelligence for IT Operations (AIOps) techniques to analyze telemetry and surface actionable incidents aligned with Service-Level Objectives (SLOs).

In production environments, incidents rarely appear as a single clean failure signal. A database slowdown, queue backlog, or downstream timeout can produce correlated metric spikes, log errors, and trace anomalies across multiple services. When these signals are handled independently, on-call teams receive duplicate or low-context alerts, and the highest business-impact incidents may not be routed first. This creates an operational gap between telemetry collection and incident response: monitoring systems must not only detect abnormal behavior, but also group related evidence, control false positives, account for SLA impact, and route alerts within human on-call capacity.

The Model Context Protocol (MCP) [6] provides a standardized interface for connecting applications and agents to live system telemetry, including metrics, logs, and traces. In this work, MCP is used as the context and orchestration layer rather than as the anomaly-detection algorithm itself. The technical contribution is the operational formulation built on top of this layer: incident correlation, controlled-FPR adaptive detection, SLA-weighted prioritization, and capacity-aware routing are combined into a reproducible multi-agent pipeline for incident management.

Specifically, we investigate the following research questions:

- **RQ1:** Can a probabilistic event-correlation model improve incident grouping accuracy and reduce duplicate alerts compared to heuristic tag-and-time overlap?
- **RQ2:** Does SLA-weighted prioritization reduce operator workload and SLA impact compared to unweighted or static routing?
- **RQ3:** Does adaptive thresholding (CUSUM with controlled FPR) improve precision and detection latency relative to fixed z-score and static thresholds?
- **RQ4:** How do correlation, SLA weighting, and adaptive thresholding interact? Which components contribute most to improvements in precision, recall, and detection latency?

Contributions. This paper makes four contributions. First, it formulates SLA-aware incident management as a utility-driven problem with constraints on false positives, mean time to detect (MTTD), and owner capacity. Second, it introduces an incremental incident-grouping model based on temporal and tag similarity with per-event work $O(K)$. Third, it combines cumulative sum (CUSUM)-based adaptive detection with SLA-weighted scoring and capacity-aware routing. Fourth, it implements these components as an MCP-orchestrated multi-

TABLE I
COMPARISON OF MONITORING SYSTEMS.

System	Data Types	Deployment	Alerting	AI/ML	SLA
Prometheus+Grafana	Metrics only	Open-source	Static rules	No built-in ML	Limited
ELK Stack	Logs/Traces	Open-source	Query-based	No native ML	No
Dynatrace Davis AI	Metrics/Logs/Traces	Commercial	AI root-cause	Proprietary	Yes (SLO)
MCP-Agent (ours)	Metrics, Logs, Traces	Framework	Multi-agent	SLA-aware scoring	Yes

agent pipeline and evaluates the design using ablations, baselines, and reproducible microservice telemetry experiments.

II. Related Work

Open-source monitoring stacks: Prometheus [1] with Grafana [2] is widely used for metrics collection in cloud-native systems. The Elasticsearch, Logstash, and Kibana (ELK) stack [3] is a de facto standard for centralized logging. Commercial application performance monitoring (APM) platforms [4], [5] integrate metrics, logs, traces and add anomaly detection. Dynatrace Davis correlates telemetry with service topology to surface impactful incidents; internals are proprietary.

Adaptive alerting and anomaly detection: The alert fatigue problem is well-documented. Sivaraman et al. [7] present an LSTM-based adaptive threshold model. Dong [8] proposes big-data architectures augmented with AI for data-center monitoring. These works show the promise of integrating AI with monitoring but stop short of a complete agent-driven framework.

Reliability and agent-based operations: Recent work has explored large-language-model-assisted root-cause analysis for microservice architectures and reliability-oriented platform design using service-level indicator (SLI) and SLO playbooks. Patel et al. [9] propose MCP-BA, a governed and explainable Model Context Protocol framework for enterprise analytics, emphasizing identity-aware reasoning, context management, governance, and auditability. Our work differs by applying MCP orchestration specifically to operational monitoring, with SLA-aware incident grouping, controlled-FPR detection, and capacity-aware alert routing within an MCP-orchestrated monitoring pipeline.

MCP and AIOps: The Model Context Protocol [6], [10] standardizes agent-tool communication. We model alert routing as score-based selection under FPR and capacity constraints, and incident grouping with per-event work $O(K)$. Table I compares related systems. Our framework combines metrics, logs, and traces with correlation, scoring, and prioritization within the MCP standard.

III. Problem Formulation

Inputs. Telemetry stream $\mathcal{T} = \{(t, m, \ell, \tau)\}$: time t, metrics m, logs ℓ, traces τ from distributed services. SLA definitions $\{(M_i, \theta_i, w_i)\}_{i=1}^{N}$: for each SLA i, metric M_i, threshold θ_i, and business weight $w_i > 0$. Service topology (optional): graph G_{svc} over services.

Outputs. Set of incidents $\mathcal{I}_1, \mathcal{I}_2, \ldots$; for each incident: anomaly score, SLA-weighted priority P, and assigned owner.

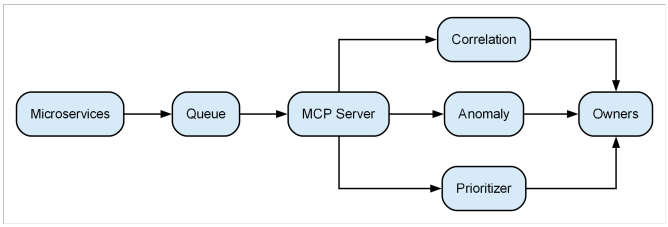

Fig. 1. Distributed monitoring architecture. Microservices emit telemetry to an MCP server. Specialized agents (correlation, anomaly scoring, prioritization) retrieve context via MCP and route alerts to device/service owners.

Fig. 2. Pipeline flowchart (Microservices → Queue → MCP Server → Correlation → Anomaly → Prioritizer → Owners).

Alert stream: incidents with $P \geq \tau$ are escalated; others logged as low-priority.

Objective. Maximize utility $\mathcal{U} = \mathbb{E}[V_{\text{esc}} - \lambda(C_{\text{FP}} + C_{\text{op}})]$, where V_{esc} is the value of timely escalation, C_{FP} is false-positive cost, and C_{op} is operator load. Equivalently: minimize expected SLA penalty plus handling cost. **Constraints:** (1) mean time to detect (MTTD) $\leq T_D$ for SLA-critical events; (2) bounded false positive rate α; (3) routing respects owner capacity (at most B alerts per time window).

IV. System Architecture

Our architecture is illustrated in Fig. 1. Microservices emit telemetry into a central MCP server. Multiple agents register as MCP clients.

Role of MCP. MCP is not treated as a new statistical detector. Its role is to standardize context access and agent coordination: the correlation agent retrieves recent telemetry context, the anomaly-scoring agent evaluates metric drift, and the prioritization component combines anomaly evidence with SLA metadata before routing. This separation allows each component to be replaced or extended while preserving the same telemetry-access and context-exchange interface.

Each agent operates autonomously. An Event-Correlation Agent groups related events into incidents. An Anomaly-Scoring Agent calculates statistical or ML-based anomaly scores. A Prioritization component weights events by business impact (via SLAs). All components communicate over MCP-defined channels.

Data flow (Fig. 2): Microservices → Data Queue → MCP Server → Event Correlation Agent → Anomaly Scoring Agent → Alert Prioritizer → Device/Service Owner. As new data arrives, agents update models and thresholds incrementally.

A. Operational Constraints

The design is shaped by operational limits. **Alert budget:** at most B alerts per on-call window to avoid overload; routing selects the top-B by SLA-weighted score. **Rate limits and backpressure:** the queue and MCP server apply backpressure when consumers are slow so that telemetry ingestion does not

overwhelm the pipeline. **Access control:** MCP access is gated by authentication and authorization so only permitted agents and users see data. **On-call noise:** we target a false positive rate (e.g., FPR/h ≤ 6 in our experiments) so that operators are not flooded with spurious alerts; CUSUM threshold h and escalation cutoff τ are tuned to meet this.

V. Design Rationale and Formalization

A. Incident Grouping Model

We model the likelihood that two events e_i, e_j belong to the same incident. Let SI denote the event that both telemetry events are part of the same incident:

$$\Pr(\text{SI} \mid e_i, e_j) \propto \exp\left(-\beta_t|t_i - t_j| - \beta_s(1 - \text{sim}(\text{tags}_i, \text{tags}_j))\right) \tag{1}$$

Here sim is Jaccard similarity on tags; the formula favors events that are close in time and have tag overlap. Per-event work scales with the number of open incidents K.

B. SLA-Weighted Scoring and Routing

We model routing as selecting up to B alerts per window to maximize total score:

$$\sum_{a \in S} \Big(\sum_{i=1}^{N} w_i f_i(a) - \lambda c(a) \Big), \quad |S| \leq B \tag{2}$$

With uniform handling cost $c(a) = c$ and fixed capacity B, selecting the top-B by score is equivalent to sorting by $u(a) = \sum_i w_i f_i(a) - \lambda c$. We use this greedy-by-score policy; when costs vary, we keep the same policy as a practical heuristic.

C. Adaptive Thresholding (CUSUM)

CUSUM [11], [12] provides drift-aware detection with a tunable false-alarm rate. The statistic is

$$S_t = \max\big(0,\ S_{t-1} + (X_t - \mu_0)/\sigma_0 - k\big), \quad \text{alert if } S_t > h \tag{3}$$

Threshold h is set to meet a target FPR; we use CUSUM in place of fixed z-score for better adaptation to baseline drift.

Implementation notes. Grouping: β_t, β_s control time and tag sensitivity. Routing: B is the on-call alert budget; τ is the escalation cutoff. CUSUM: k is the drift allowance, h the alert threshold; both are tuned on validation (evaluation section).

VI. SLA-Weighted Alert Prioritization

Normalized violation for SLA i: $f_i(E) = (v_i(E) - \theta_i)_+ / \sigma_i$. Priority score:

$$P(E) = \sum_{i=1}^{N} w_i \max\big(0, f_i(E)\big) \tag{4}$$

If $P(E) \geq \tau$, escalate; otherwise log as low-priority.

TABLE II
Track A: KPI anomaly detection (point-level, mean ± 95% CI, 30 runs). FP/h = false alerts/hour.

Method	Prec.	Rec.	F1	FP/h	Delay
Static	0.933 ± 0.067	0.155 ± 0.078	0.243 ± 0.107	0.0001 ± 0.0001	4.13 ± 2.30
Z-score	0.620 ± 0.094	0.109 ± 0.026	0.180 ± 0.037	0.473 ± 0.134	0.20 ± 0.33
CUSUM	0.444 ± 0.063	0.261 ± 0.038	0.319 ± 0.032	2.380 ± 0.267	1.50 ± 1.03
MCP-Agent	0.444 ± 0.064	0.261 ± 0.035	0.319 ± 0.033	2.380 ± 0.280	1.50 ± 1.05

VII. Algorithms

Event correlation: For each event e, compute $\Pr(\text{same incident} \mid e, I)$ for each open incident I; assign e to the highest-probability incident if above threshold, else open a new incident. Per-event work: $O(K)$.

CUSUM: Maintain $S_t = \max(0, S_{t-1} + (x_t - \mu_0)/\sigma_0 - k)$; alert when $S_t > h$. $O(1)$ per sample.

Alert routing: Compute $P(I) = \sum_i w_i \max(0, f_i(I))$; if $P(I) \geq \tau$, notify owner; else log. $O(N)$ per incident.

Pipeline: Correlation → Anomaly → Prioritization & Routing.

VIII. Experimental Design and Evaluation

Datasets: Public traces or reproducible synthetic telemetry ($N \approx 100$ services). **Baselines:** Static threshold, z-score, tag+time correlation, and Bayesian changepoint detection. **Metrics:** Precision, recall, F1 at incident level; FPR per hour; MTTD; mean time to resolve (MTTR). **Statistical rigor:** 30 runs, mean ± 95% bootstrap CI; Wilcoxon, Holm correction; Cliff's delta.

Metric definitions. Track A (point-level): Precision = TP/(TP+FP), Recall = TP/(TP+FN), F1 = 2PR/(P+R); FP/h = FP per test-hour; detection delay in timesteps. Track C (incident-level): intersection-over-union (IoU) bipartite matching; FPR/h = false alerts per hour.

Fair tuning. Track C: tune to FPR/h ≤ 6. Track A: thresholds via grid search on validation.

A. Evaluation on Public KPI Dataset (Track A)

We use a public KPI benchmark from the AIOps 2018 challenge [13] (2000 points, 70/30 split, labeled anomalies) as a sanity check on anomaly detection only. Track A does not exercise correlation or routing (those require multi-source telemetry); Track A isolates anomaly detection; in this setting MCP-Agent reduces to CUSUM.

B. Evaluation on Microservices Telemetry (Track C)

Track C uses OTel-style simulator with chaos-injected faults and incident-level IoU matching. Table III summarizes the results.

Statistical tests (Wilcoxon, Holm): MCP-Agent vs Static — precision $p < 0.001$ (MCP higher), recall $p = 0.0002$ (Static higher); MCP vs Tag+time — F1 $p < 0.001$ (MCP higher). Cliff's delta (MCP vs Static recall) $= -0.53$.

Key findings. In Track C (Table III), MCP-Agent achieves precision 0.171 vs 0.055 (static) and F1 0.282 vs 0.104

TABLE III
TRACK C: INCIDENT-LEVEL EVALUATION (MEAN ± 95% CI, 30 RUNS).

Method	Prec.	Recall	F1	FPR/h	MTTD	MTTR	Alerts
MCP-Agent	0.171 ± 0.011	0.823 ± 0.033	0.282 ± 0.015	4.91 ± 0.29	2.35 ± 0.40	7.35 ± 0.38	49.2 ± 2.6
Static	0.055 ± 0.005	0.923 ± 0.027	0.103 ± 0.009	20.44 ± 1.99	2.10 ± 0.28	7.10 ± 0.31	179.6 ± 17.7
Z-score	0.038 ± 0.001	0.977 ± 0.017	0.073 ± 0.002	29.79 ± 0.84	2.01 ± 0.36	7.01 ± 0.36	258.0 ± 7.3
Tag+time	0.139 ± 0.039	0.083 ± 0.027	0.104 ± 0.030	0.56 ± 0.03	3.80 ± 2.45	6.97 ± 3.02	5.5 ± 0.3
Bayesian changepoint (CP)	0.023 ± 0.030	0.017 ± 0.020	0.019 ± 0.024	0.30 ± 0.11	0.87 ± 1.50	1.37 ± 2.17	2.7 ± 1.0

(Tag+time), with FPR 4.91/h (under the 6/h target). MCP-Agent outperforms the Bayesian changepoint baseline [14] on precision, recall, and F1. Fig. 3 shows the FPR–recall trade-off across methods.

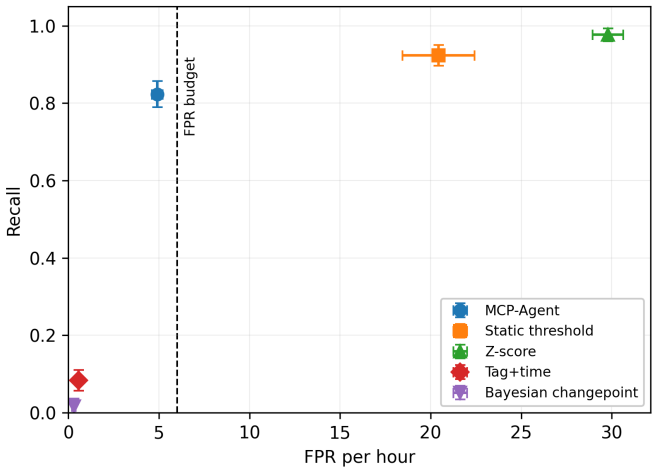

Fig. 3. Track C: FPR/h vs recall (mean ± 95% CI). The dashed line marks the target FPR budget.

Table IV and Fig. 4 summarize the ablation: full MCP-Agent gives the best precision; Tag+time and static-only configurations show the contribution of correlation and SLA-weighted routing.

TABLE IV
ABLATION: PRECISION AND FPR BY CONFIGURATION (TRACK C, 30 RUNS).

Config	Precision	FPR/h	Δ vs full
Full MCP-Agent	0.171	4.91	baseline
Static only	0.055	20.44	−0.116
Tag+time only	0.139	0.56	−0.032
Bayesian changepoint	0.023	0.30	−0.148

C. Scalability

We vary the number of services (10, 50, 100, 200) and measure per-event latency (p50, p99) and throughput for the MCP-Agent pipeline over multiple seeds (mean ± 95% CI). Results are in Table V and Fig. 5. p50 latency stays below 0.35 ms and throughput above 1800 events/s up to 200 services. We did not run 500 services due to run-time limits.

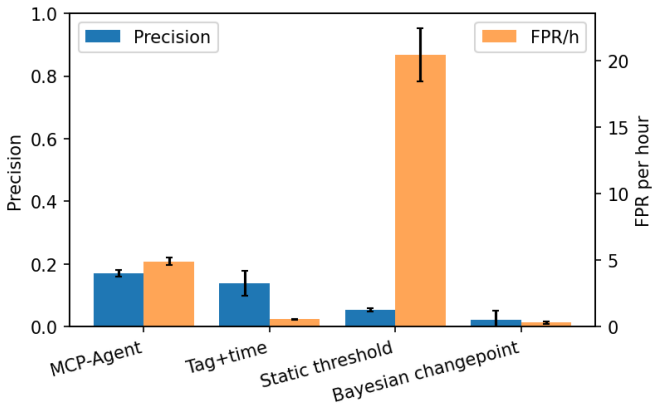

Fig. 4. Ablation: precision and FPR/h by configuration.

TABLE V
SCALABILITY: LATENCY AND THROUGHPUT VS NUMBER OF SERVICES.

Services	p50 (ms)	p99 (ms)	Throughput (ev/s)
10	0.31 ± 0.00	1.13 ± 0.15	3121.9 ± 128.0
50	0.32 ± 0.00	3.88 ± 0.13	2220.3 ± 108.7
100	0.32 ± 0.00	2.96 ± 0.17	2311.5 ± 77.4
200	0.34 ± 0.00	3.72 ± 0.06	1853.1 ± 54.1

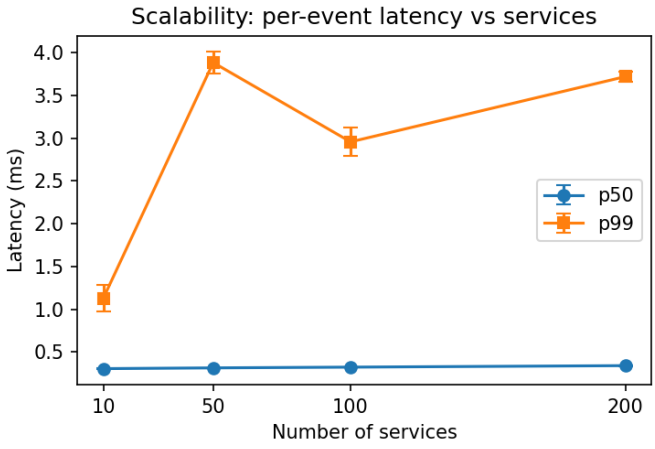

Fig. 5. Scalability: per-event latency (p50, p99) vs services.

IX. Discussion, Limitations, and Future Work

Effectiveness depends on telemetry quality, SLA metadata, and appropriate validation-time tuning of thresholds. Scalability is reported up to 200 services (Table V, Fig. 5); we did not run 500 services due to run-time limits. The evaluation demonstrates the effectiveness of the full MCP-agent pipeline, but isolating MCP protocol orchestration against an equivalent non-MCP implementation remains future work. Security, including authentication and access-control lists (ACLs), is also critical for production deployment. Future work includes agent-to-agent protocols, learning SLA weights from historical incident outcomes, and integration with incident management tools [15].

X. Conclusion

We presented an SLA-aware incident detection and routing framework integrating correlation, adaptive thresholding, and SLA-weighted prioritization. Evaluation shows improved precision under FPR constraints while maintaining detection latency and scalability.

Acknowledgments

We thank the developers of MCP, Prometheus, and open data communities for enabling this research.

References

[1] Prometheus Authors, "Prometheus," https://prometheus.io, 2024, accessed: Jun. 20, 2026.

[2] Grafana Labs, "Grafana," https://grafana.com, 2024, accessed: Jun. 20, 2026.

[3] Elastic, "Elastic Stack," https://www.elastic.co/elastic-stack, 2024, accessed: Jun. 20, 2026.

[4] Datadog, "Datadog," https://www.datadoghq.com, 2024, accessed: Jun. 20, 2026.

[5] Dynatrace, "Dynatrace Davis AI," https://www.dynatrace.com/platform/davis-ai, 2024, accessed: Jun. 20, 2026.

[6] Anthropic, "Introducing the Model Context Protocol," https://www.anthropic.com/news/model-context-protocol, 2024, accessed: Jun. 20, 2026.

[7] V. Sivaraman, "LSTM-Based Adaptive Threshold Model for Reducing False Positives in Monitoring," in *Proceedings of AIOps/ML for Systems*, 2022.

[8] W. Dong, "Big-Data Architectures Augmented with AI for Data-Center Monitoring," *IEEE Transactions on Cloud Computing*, 2022.

[9] T. P. Patel, S. Shivam, A. K. Padhy, B. Vulugunda, C. Kulkarni, and C. Medicherla, "Model Context Protocol Business Analyst (MCP-BA): A Governed and Explainable Framework for Enterprise Analytics: Operationalizing Identity-Aware Reasoning and Context-Driven Decision Intelligence in Enterprise AI," in *2025 International Conference on Computer and Applications (ICCA)*, Bahrain, Bahrain, 2025, pp. 1–7.

[10] Model Context Protocol, "Model Context Protocol Documentation," https://modelcontextprotocol.io/docs/getting-started/intro, 2026, accessed: Jun. 20, 2026.

[11] E. S. Page, "Continuous Inspection Schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.

[12] G. Lorden, "Procedures for Reacting to a Change in Distribution," *The Annals of Mathematical Statistics*, vol. 42, no. 6, pp. 1897–1908, 1971.

[13] H. Guo *et al.*, "AIOps 2018 Challenge: Intelligent Anomaly Detection in KPI," in *ACM KDD Workshop on AIOps*, 2018.

[14] C. Truong, L. Oudre, and N. Vayatis, "Selective Review of Offline Change Point Detection Methods," *Signal Processing*, vol. 167, p. 107299, 2020.

[15] PagerDuty, "PagerDuty," https://www.pagerduty.com, 2024, accessed: Jun. 20, 2026.